

Algorithms

Divide and Conquer

Dr. Mudassir Shabbir

LUMS University

March 4, 2026

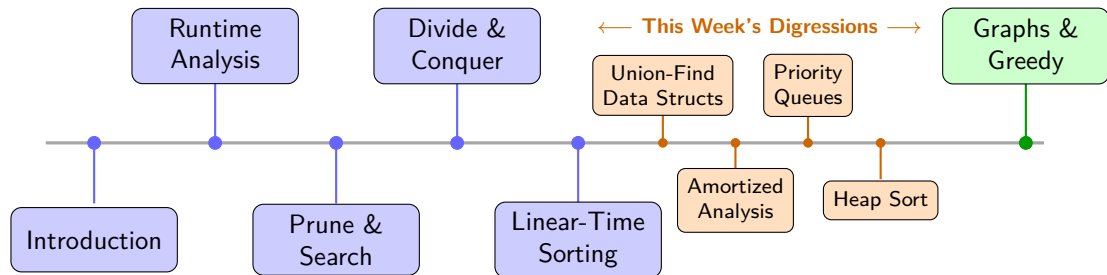


Announcements

- HW 3 is due on [Feb 09](#) at **11:59 AM**.
- HW 4 is out - no submission required.



Recap & What's Next



Minimum Spanning Tree



Why Minimum Spanning Trees? (Historical Motivation)

- During World War II, the Allies controlled railway infrastructure across Europe.
- Maintaining rail lines was expensive: fuel, guards, repairs, signaling.
- They needed a network that:
 - keeps all cities connected,
 - minimizes total maintenance cost.
- Question: **Which tracks should remain open?**



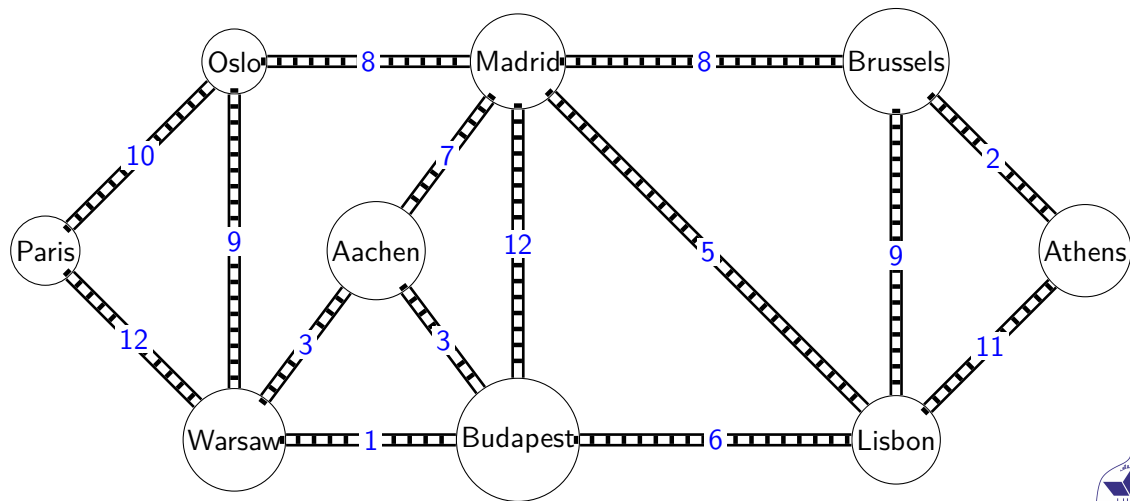
Why Minimum Spanning Trees? (Historical Motivation)

- During World War II, the Allies controlled railway infrastructure across Europe.
- Maintaining rail lines was expensive: fuel, guards, repairs, signaling.
- They needed a network that:
 - keeps all cities connected,
 - minimizes total maintenance cost.
- Question: **Which tracks should remain open?**

This is exactly the Minimum Spanning Tree problem.

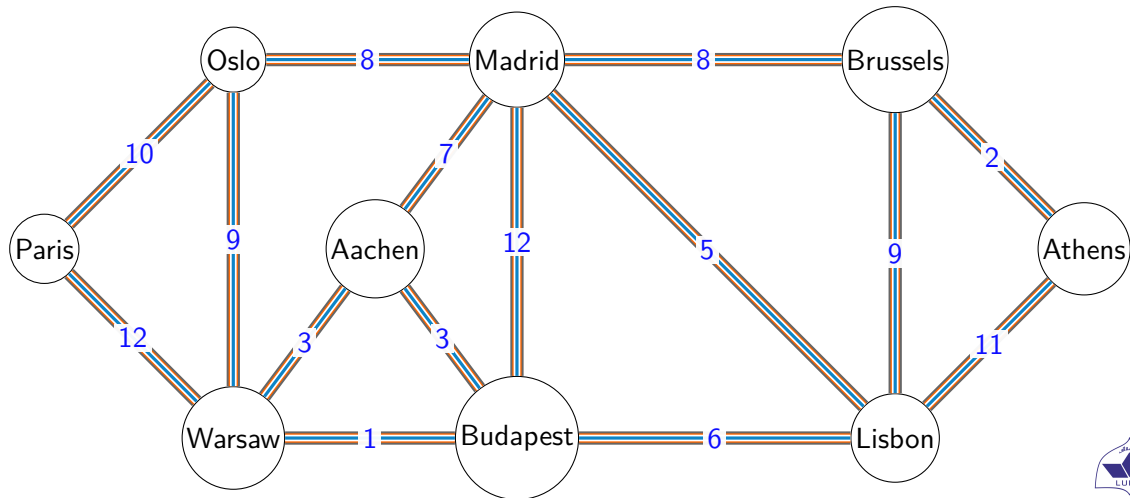


European rail network

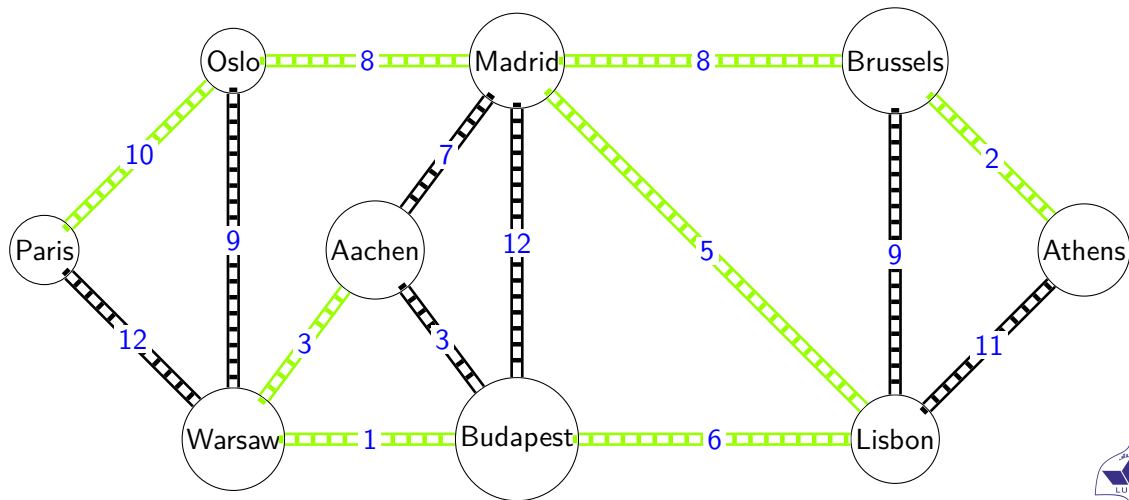


Goal: Keep cities connected while minimizing total cost.

A Modern Version: Fiber Network Design



Example Minimum Spanning Tree



Minimum Spanning Tree (MST) Problem

- Input: Weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}^+$.
- Output: Subset of edges $T \subseteq E$ such that:
 - T connects all vertices (spanning),
 - T contains no cycles (tree),
 - T has minimum total weight: $\sum_{e \in T} w(e)$.



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.

Question: How many subsets of edges are there?



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.

Question: How many subsets of edges are there?

$$2^{|E|}$$



How would we compute this?

- Idea: Try all subsets of edges.
- Keep only those that:
 - connect all vertices,
 - contain no cycle.
- Choose the cheapest among them.

Question: How many subsets of edges are there?

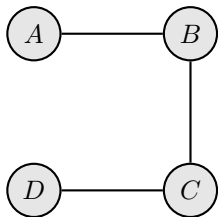
$$2^{|E|}$$

Conclusion: Brute force is exponential — infeasible.



Basic Fact 1: Adding an Edge to a Tree Creates a Cycle

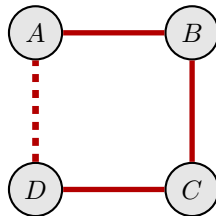
Tree (no cycle)



3 edges, 4 nodes. No cycle.

Unique path between every pair.

Add edge AD



Cycle $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$!

Now two paths exist between A and D .

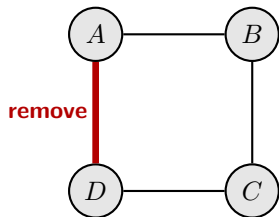
Fact. Adding *any* edge to a tree creates *exactly one* cycle.

The new edge provides a second path between its two endpoints, closing exactly one loop.



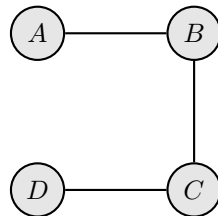
Basic Fact 2: Removing an Edge from a Cycle Keeps it Connected

Graph with cycle $A-B-C-D$



Remove edge DA .

Still connected!

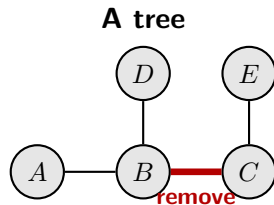


All nodes still reachable.
The result is a tree.

Fact. Removing any edge from a cycle leaves the graph *still connected*.
Every path that used that edge can be re-routed the other way around the cycle.

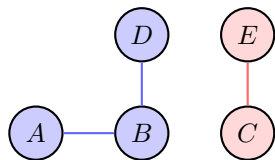


Basic Fact 3: Removing an Edge from a Tree Disconnects It



Remove edge BC .

Two components!



$\{A, B, D\}$ and $\{C, E\}$
are now disconnected.

Fact. Removing any edge from a tree creates *exactly two* connected components.

In a tree there is exactly one path between every pair of nodes. Remove the edge — all nodes on one side lose their only route to the other.

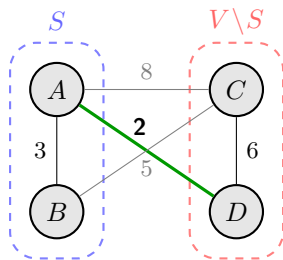


Key Insight: The Cut Property

Def. A **cut** $(S, V \setminus S)$ partitions vertices into two non-empty sets. An edge **crosses** the cut if its endpoints lie in different parts. Also called **cut edge**.

Cut Property. The unique min-weight cut edge belongs to every MST.

(Proof: same exchange argument — swap the cut edge f in the MST for e^* .)



Crossing edges: AC (8), AD (2), BC (5).

Min crossing edge: AD (weight 2).

AD must be in every MST.

Kruskal's Algorithm

```
procedure Kruskal( $G = (V, E)$ ,  $w : E \rightarrow \mathbb{R}$ )  
   $T \leftarrow \emptyset$   
  Sort edges in  $E$  by increasing weight  
  for each edge  $(u, v) \in E$  (in sorted order) do  
    if  $T \cup \{(u, v)\}$  is acyclic then  
       $T \leftarrow T \cup \{(u, v)\}$   
  return  $T$ 
```



Kruskal's Algorithm

```
procedure Kruskal( $G = (V, E)$ ,  $w : E \rightarrow \mathbb{R}$ )  
   $T \leftarrow \emptyset$   
  Sort edges in  $E$  by increasing weight  
  for each edge  $(u, v) \in E$  (in sorted order) do  
    if  $T \cup \{(u, v)\}$  is acyclic then  
       $T \leftarrow T \cup \{(u, v)\}$   
  return  $T$ 
```

Greedy algorithm: makes locally optimal choice at each step.



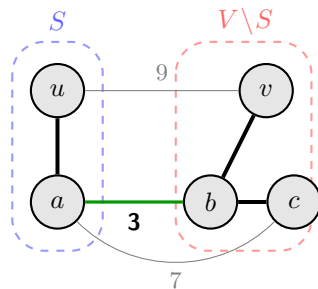
Kruskal's Correctness: Cut Property in Action

Observation. Each edge Kruskal's accepts is the cheapest edge crossing some cut.

When Kruskal's considers edge $e = (u, v)$ across two components:

- Let S = current component of u .
- e is the *cheapest* edge crossing $(S, V \setminus S)$.
(All cheaper edges were processed: accepted into S , or rejected because they stayed inside S .)
- **Cut Property** $\Rightarrow e$ belongs to some MST.
- So accepting e is always safe.

Conclusion. Kruskal's builds the MST one safe edge at a time. Every greedy choice is justified by the Cut Property.

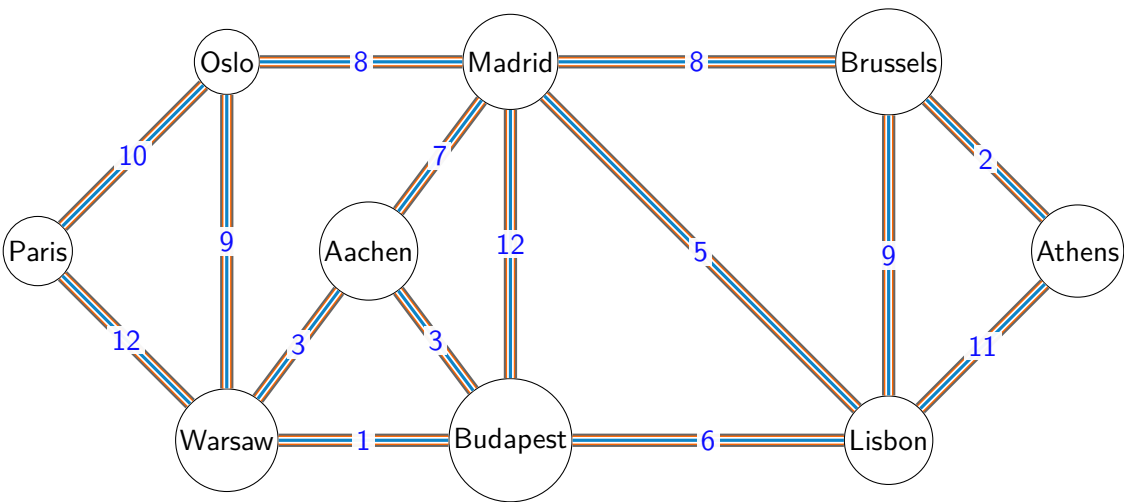


ab (weight 3): cheapest crossing edge.

Cut Property $\Rightarrow$ safe to accept.

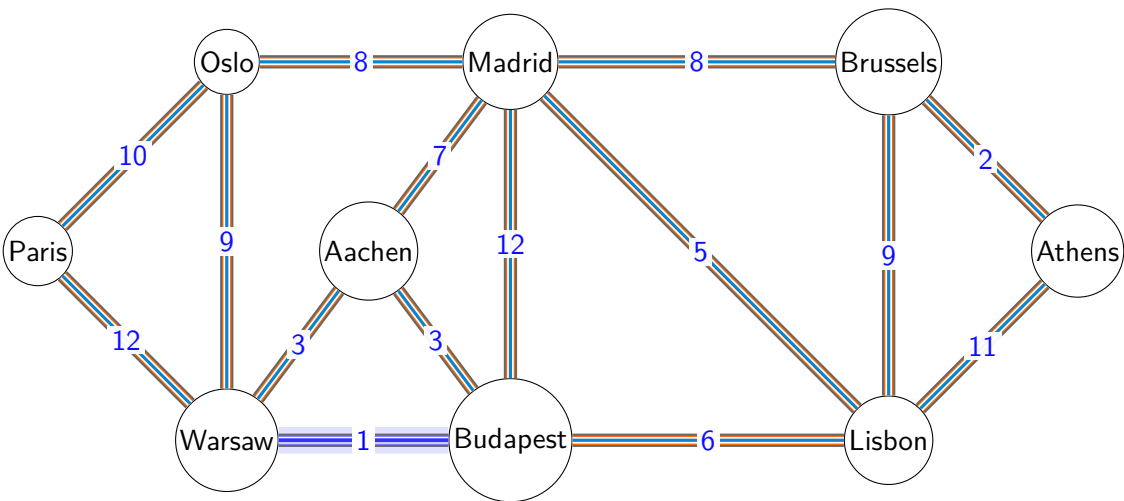


Let's Run Kruskal's Algorithm!



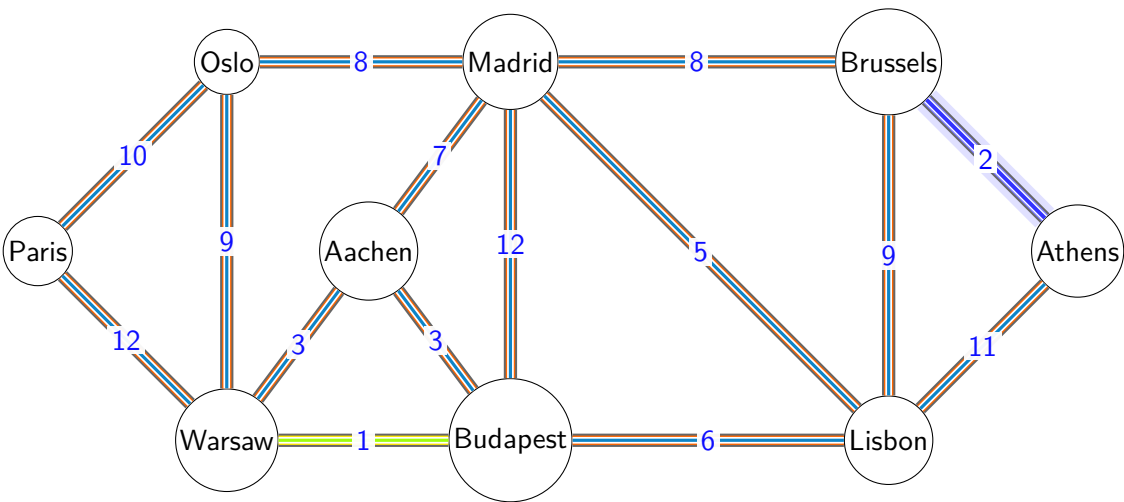
Testing edge CF (weight 1)

Accept - No cycle created



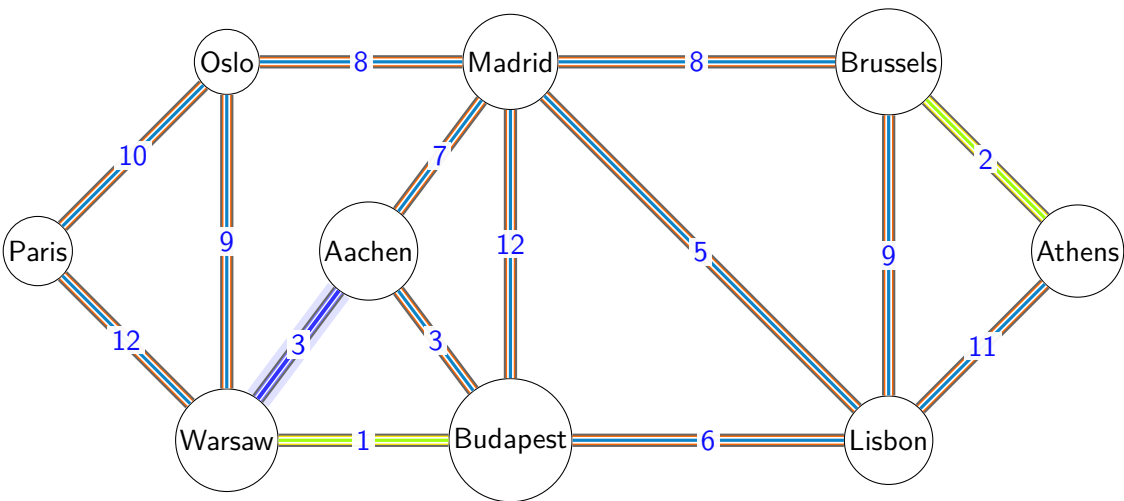
Testing edge GI (weight 2)

Accept - No cycle created

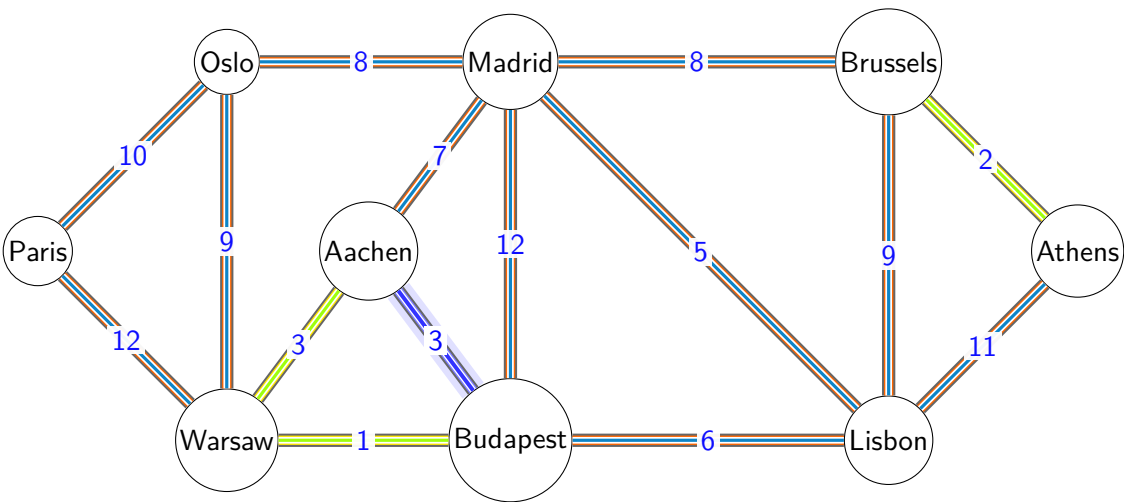


Testing edge CE (weight 3)

Accept - No cycle created

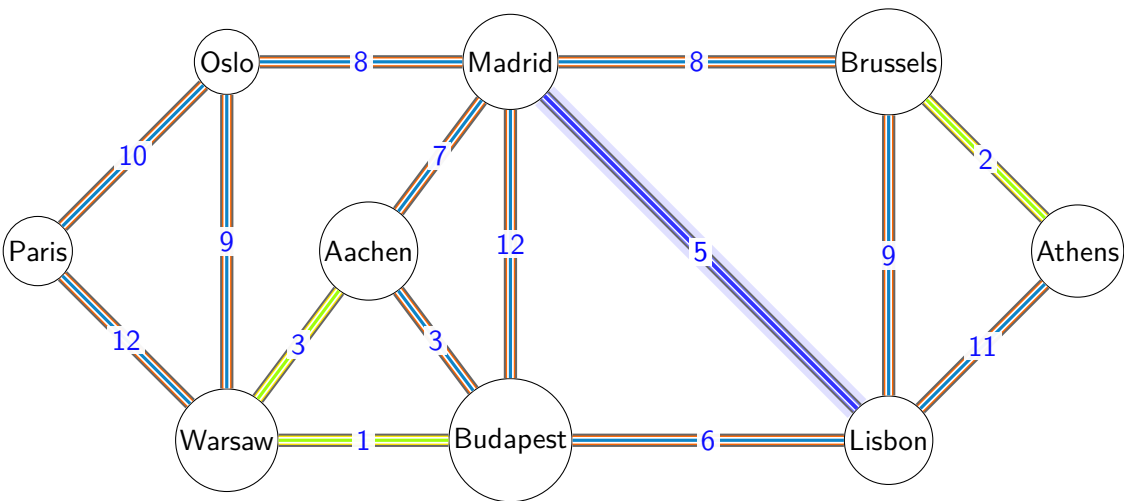


Testing edge EF (weight 3)
Reject - Creates cycle: C-E-F-C



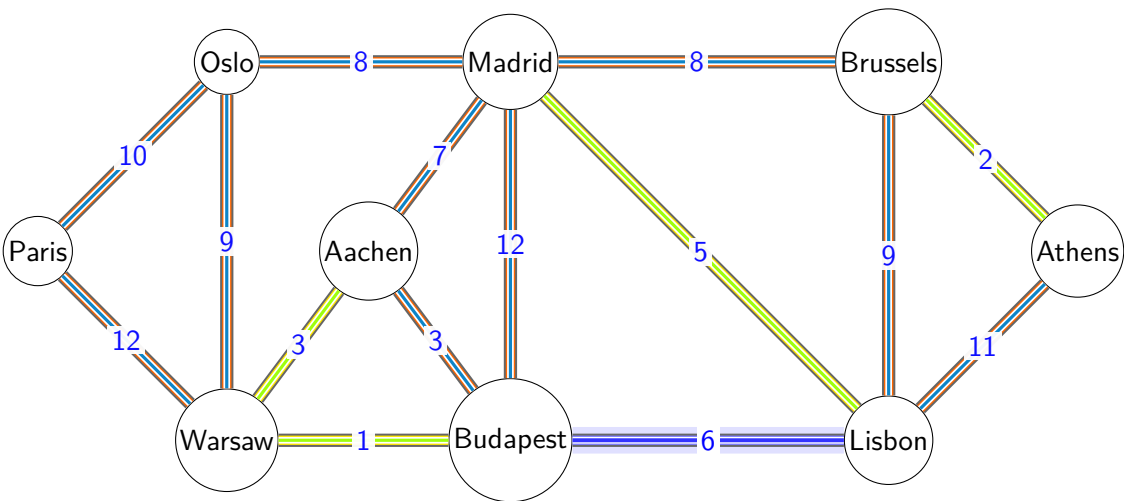
Testing edge DH (weight 5)

Accept - No cycle created



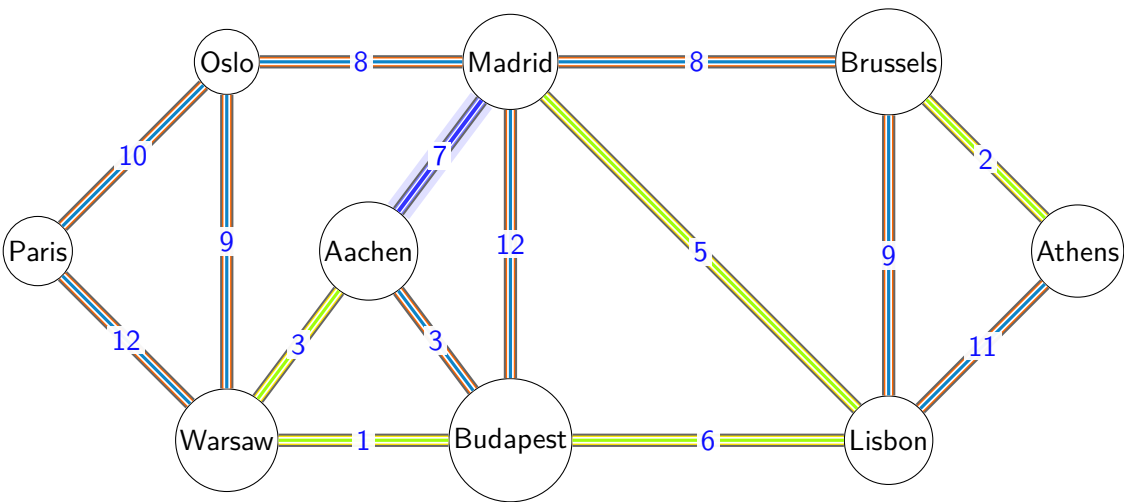
Testing edge FH (weight 6)

Accept - Connects two components



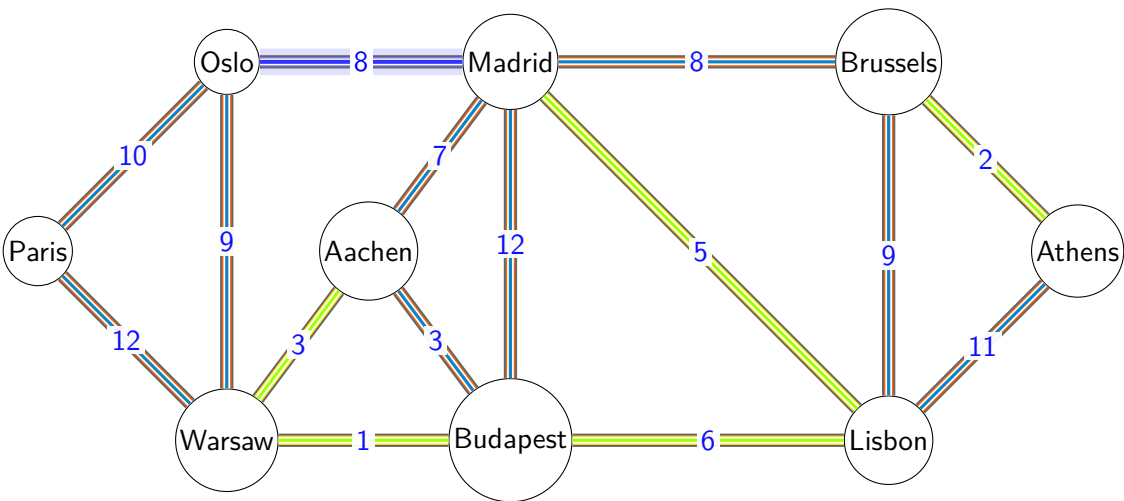
Testing edge DE (weight 7)

Reject - Creates cycle: D-H-F-C-E-D



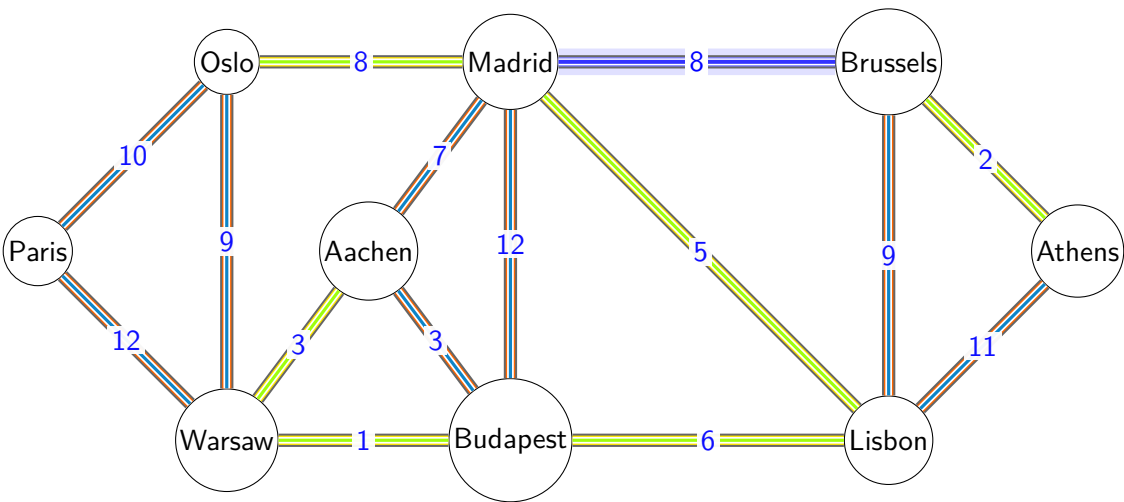
Testing edge BD (weight 8)

Accept - No cycle created



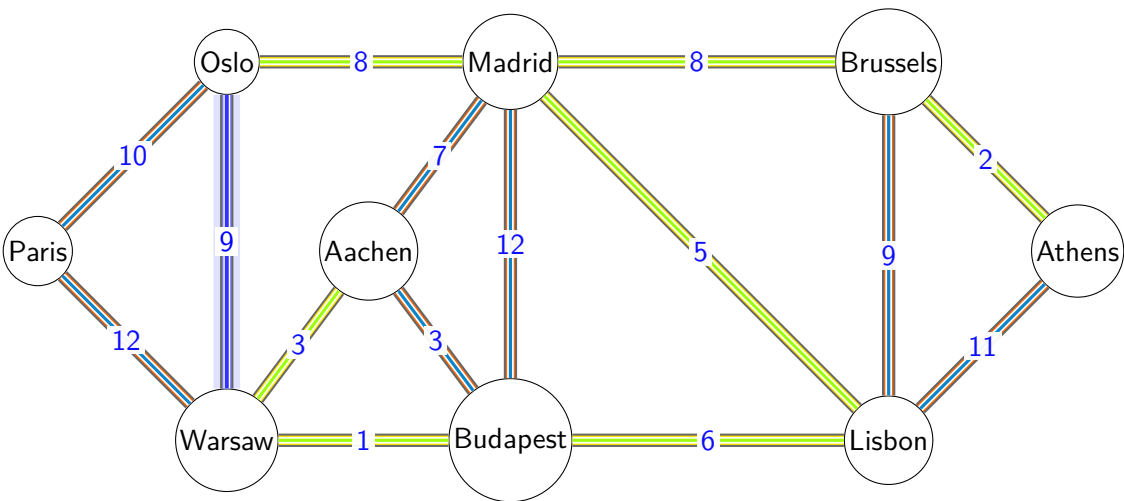
Testing edge DG (weight 8)

Accept - Connects components



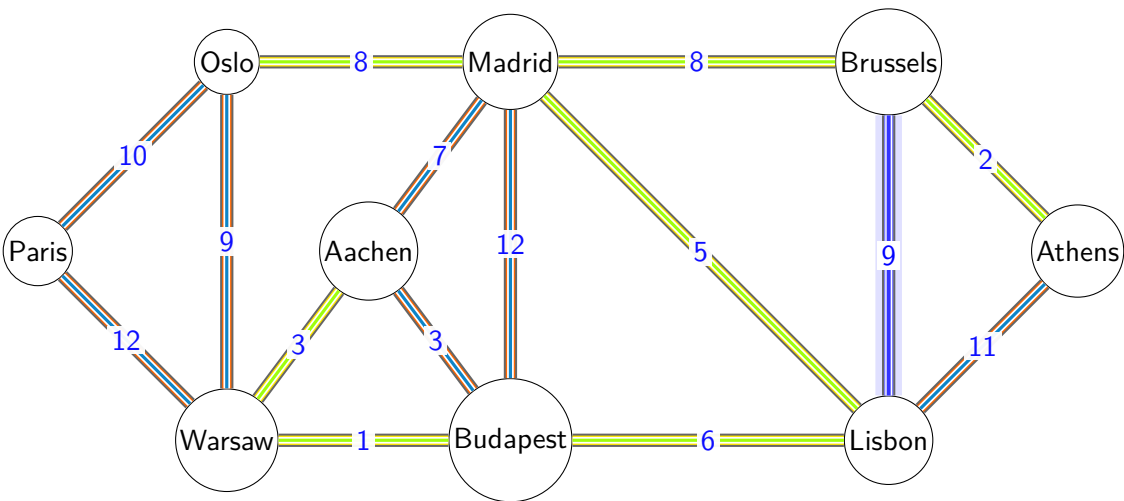
Testing edge BC (weight 9)

Reject - Creates cycle



Testing edge GH (weight 9)

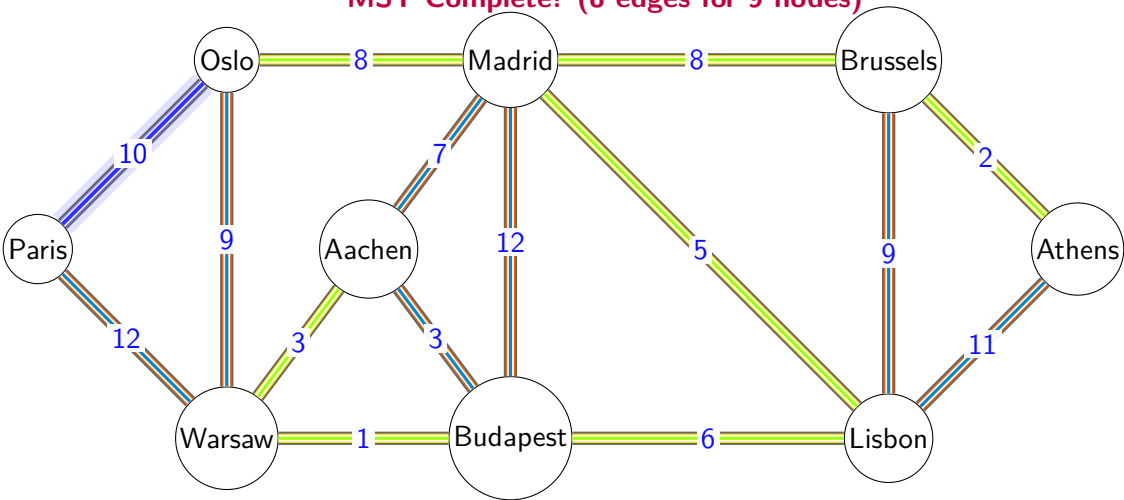
Reject - Creates cycle



Testing edge AB (weight 10)

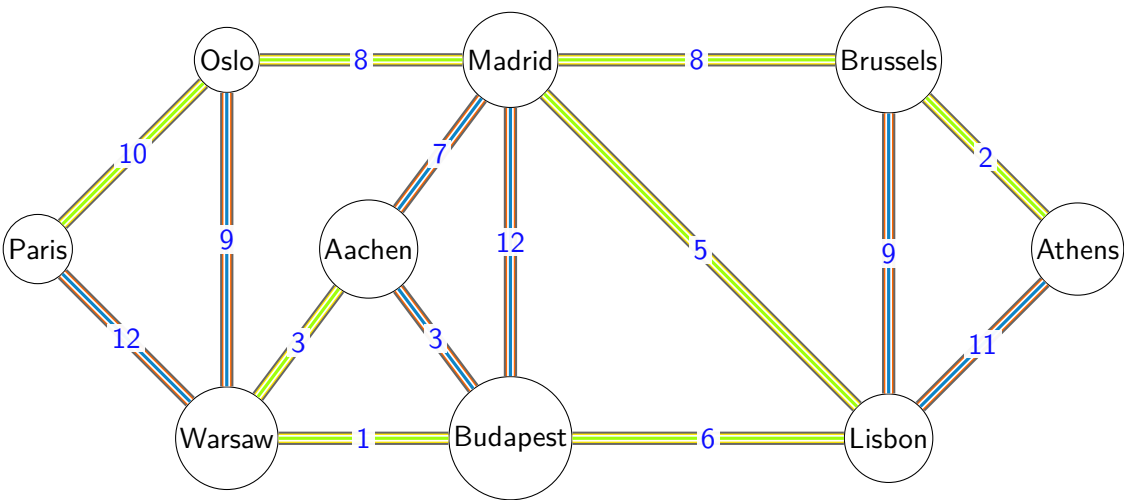
Accept - No cycle created

MST Complete! (8 edges for 9 nodes)



Minimum Spanning Tree

Total Weight: $1+2+3+5+6+8+8+10 = 43$



Kruskal's Complexity

Step 1. Sort all $|E|$ edges by weight.



Kruskal's Complexity

Step 1. Sort all $|E|$ edges by weight. $O(E \log E)$



Kruskal's Complexity

Step 1. Sort all $|E|$ edges by weight. $O(E \log E)$

Step 2. For each edge: check if it creates a cycle

Before adding edge (u, v) : run DFS/BFS from u in the current forest. If v is reachable $\Rightarrow$ cycle; reject. Otherwise accept



Kruskal's Complexity

Step 1. Sort all $|E|$ edges by weight. $O(E \log E)$

Step 2. For each edge: check if it creates a cycle

Before adding edge (u, v) : run DFS/BFS from u in the current forest. If v is

reachable $\Rightarrow$ cycle; reject. Otherwise accept $O(V)$ per edge



Kruskal's Complexity

Step 1. Sort all $|E|$ edges by weight. $O(E \log E)$

Step 2. For each edge: check if it creates a cycle

Before adding edge (u, v) : run DFS/BFS from u in the current forest. If v is reachable $\Rightarrow$ cycle; reject. Otherwise accept $O(V)$ per edge

Total Cost. $|E|$ edges $\times O(V)$ each.



Kruskal's Complexity

Step 1. Sort all $|E|$ edges by weight. $O(E \log E)$

Step 2. For each edge: check if it creates a cycle

Before adding edge (u, v) : run DFS/BFS from u in the current forest. If v is

reachable $\Rightarrow$ cycle; reject. Otherwise accept $O(V)$ per edge

Total Cost. $|E|$ edges $\times O(V)$ each. $O(EV)$



Kruskal's Complexity

Step 1. Sort all $|E|$ edges by weight. $O(E \log E)$

Step 2. For each edge: check if it creates a cycle

Before adding edge (u, v) : run DFS/BFS from u in the current forest. If v is reachable $\Rightarrow$ cycle; reject. Otherwise accept $O(V)$ per edge

Total Cost. $|E|$ edges $\times O(V)$ each. $O(EV)$

Part	Cost
Sort edges	$O(E \log E)$
All cycle checks	$O(EV)$
Total	$O(EV)$

Sparse ($E = O(V)$): $O(V^2)$

Dense ($E = O(V^2)$): $O(V^3)$



Kruskal's Complexity: Using Union-Find

Replace DFS/BFS with a Union-Find (Disjoint Set Union) structure.



Kruskal's Complexity: Using Union-Find

Replace DFS/BFS with a Union-Find (Disjoint Set Union) structure.

Two operations:

- $\text{Find}(x)$: which component is x in?
- $\text{Union}(x, y)$: merge two components.



Kruskal's Complexity: Using Union-Find

Replace DFS/BFS with a Union-Find (Disjoint Set Union) structure.

Two operations:

- $\text{Find}(x)$: which component is x in?
- $\text{Union}(x, y)$: merge two components.

With *path compression* + *union by rank*:

- $O(\alpha(V))$ amortized per operation.
- α = inverse Ackermann; ≤ 4 in practice.



Kruskal's Complexity: Using Union-Find

Replace DFS/BFS with a Union-Find (Disjoint Set Union) structure.

Two operations:

- $\text{Find}(x)$: which component is x in?
- $\text{Union}(x, y)$: merge two components.

With *path compression* + *union by rank*:

- $O(\alpha(V))$ amortized per operation.
- α = inverse Ackermann; ≤ 4 in practice.

Kruskal's per edge (u, v) :

- 1 $\text{Find}(u), \text{Find}(v)$ — same component?
- 2 If same: reject. If different: $\text{Union}(u, v)$.



Kruskal's Complexity: Using Union-Find

Replace DFS/BFS with a Union-Find (Disjoint Set Union) structure.

Two operations:

- $\text{Find}(x)$: which component is x in?
- $\text{Union}(x, y)$: merge two components.

With *path compression* + *union by rank*:

- $O(\alpha(V))$ amortized per operation.
- α = inverse Ackermann; ≤ 4 in practice.

Kruskal's per edge (u, v) :

- 1 $\text{Find}(u), \text{Find}(v)$ — same component?
- 2 If same: reject. If different: $\text{Union}(u, v)$.

$$\underbrace{O(E \log E)}_{\text{sort}} + \underbrace{O(E \cdot \alpha(V))}_{\text{Union-Find ops}} = \mathbf{O(E \log E)}$$



Kruskal's Complexity: Using Union-Find

Replace DFS/BFS with a Union-Find (Disjoint Set Union) structure.

Two operations:

- $\text{Find}(x)$: which component is x in?
- $\text{Union}(x, y)$: merge two components.

With *path compression* + *union by rank*:

- $O(\alpha(V))$ amortized per operation.
- α = inverse Ackermann; ≤ 4 in practice.

Kruskal's per edge (u, v) :

- 1 $\text{Find}(u), \text{Find}(v)$ — same component?
- 2 If same: reject. If different: $\text{Union}(u, v)$.

$$\underbrace{O(E \log E)}_{\text{sort}} + \underbrace{O(E \cdot \alpha(V))}_{\text{Union-Find ops}} = \mathbf{O(E \log E)}$$

Graph	$ E $	DFS/BFS	Union-Find
Sparse	$O(V)$	$O(V^2)$	$O(V \log V)$
Dense	$O(V^2)$	$O(V^3)$	$O(V^2 \log V)$



Kruskal's vs Prim's: Two Greedy Strategies

Kruskal's builds a *forest*.

- Sort all edges globally upfront.
- Process edges in weight order.
- Merge two components with each accepted edge.
- The forest becomes connected only at the end.



Kruskal's vs Prim's: Two Greedy Strategies

Kruskal's builds a *forest*.

- Sort all edges globally upfront.
- Process edges in weight order.
- Merge two components with each accepted edge.
- The forest becomes connected only at the end.

Prim's grows a single *connected* tree.

- No global sort needed.
- Grow one tree from a start vertex.
- At each step, add the cheapest edge leaving the tree.
- The tree is connected throughout.



Kruskal's vs Prim's: Two Greedy Strategies

Kruskal's builds a *forest*.

- Sort all edges globally upfront.
- Process edges in weight order.
- Merge two components with each accepted edge.
- The forest becomes connected only at the end.

Prim's grows a single *connected* tree.

- No global sort needed.
- Grow one tree from a start vertex.
- At each step, add the cheapest edge leaving the tree.
- The tree is connected throughout.

Both algorithms are correct by the **Cut Property**: the cheapest edge crossing any cut belongs to every MST, so every greedy choice is safe.



Kruskal's vs Prim's: Two Greedy Strategies

Kruskal's builds a *forest*.

- Sort all edges globally upfront.
- Process edges in weight order.
- Merge two components with each accepted edge.
- The forest becomes connected only at the end.

Prim's grows a single *connected* tree.

- No global sort needed.
- Grow one tree from a start vertex.
- At each step, add the cheapest edge leaving the tree.
- The tree is connected throughout.

Both algorithms are correct by the **Cut Property**: the cheapest edge crossing any cut belongs to every MST, so every greedy choice is safe.

The key difference is the data structure used to find the next safe edge.

- Kruskal's: sort once upfront, then Union-Find for cycle detection.
- Prim's: a priority queue that is updated as the tree grows.



Prim's Algorithm

Maintain for each non-tree vertex v :

- $\text{key}[v]$: weight of the lightest known edge from v to the current tree (∞ if none exists).
- $\text{parent}[v]$: the tree endpoint of that edge.



Prim's Algorithm

Maintain for each non-tree vertex v :

- $\text{key}[v]$: weight of the lightest known edge from v to the current tree (∞ if none exists).
- $\text{parent}[v]$: the tree endpoint of that edge.

A **min-priority queue** Q holds all non-tree vertices keyed by $\text{key}[v]$.



Prim's Algorithm

Maintain for each non-tree vertex v :

- $\text{key}[v]$: weight of the lightest known edge from v to the current tree (∞ if none exists).
- $\text{parent}[v]$: the tree endpoint of that edge.

A **min-priority queue** Q holds all non-tree vertices keyed by $\text{key}[v]$.

Steps:

- 1 $\text{key}[s] \leftarrow 0$; $\text{key}[v] \leftarrow \infty$ for $v \neq s$.
- 2 Insert all vertices into Q .
- 3 While $Q \neq \emptyset$:
 - $u \leftarrow \text{ExtractMin}(Q)$
 - For each neighbor v of u :
if $v \in Q$ and $w(u, v) < \text{key}[v]$:
 - $\text{parent}[v] \leftarrow u$
 - $\text{key}[v] \leftarrow w(u, v)$



Prim's Algorithm

Maintain for each non-tree vertex v :

- $\text{key}[v]$: weight of the lightest known edge from v to the current tree (∞ if none exists).
- $\text{parent}[v]$: the tree endpoint of that edge.

A **min-priority queue** Q holds all non-tree vertices keyed by $\text{key}[v]$.

The MST edges are $\{(\text{parent}[v], v) : v \neq s\}$.

Steps:

- 1 $\text{key}[s] \leftarrow 0$; $\text{key}[v] \leftarrow \infty$ for $v \neq s$.
- 2 Insert all vertices into Q .
- 3 While $Q \neq \emptyset$:
 - $u \leftarrow \text{ExtractMin}(Q)$
 - For each neighbor v of u :
if $v \in Q$ and $w(u, v) < \text{key}[v]$:
 - $\text{parent}[v] \leftarrow u$
 - $\text{key}[v] \leftarrow w(u, v)$



Prim's Correctness: Cut Property at Every Step

At any point, let T be the set of vertices already in the tree.



Prim's Correctness: Cut Property at Every Step

At any point, let T be the set of vertices already in the tree.

Define the cut $(T, V \setminus T)$.

Prim's extracts the vertex $v^* = \arg \min_{v \notin T} \text{key}[v]$.

The edge $(\text{parent}[v^*], v^*)$ is the *minimum weight edge crossing this cut*.



Prim's Correctness: Cut Property at Every Step

At any point, let T be the set of vertices already in the tree.

Define the cut $(T, V \setminus T)$.

Prim's extracts the vertex $v^* = \arg \min_{v \notin T} \text{key}[v]$.

The edge $(\text{parent}[v^*], v^*)$ is the *minimum weight edge crossing this cut*.

Cut Property $\Rightarrow$ this edge belongs to every MST.

Every greedy step is safe. *Prim's is correct.*



Prim's Correctness: Cut Property at Every Step

At any point, let T be the set of vertices already in the tree.

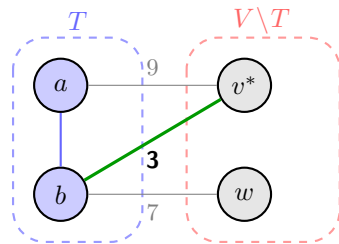
Define the cut $(T, V \setminus T)$.

Prim's extracts the vertex $v^* = \arg \min_{v \notin T} \text{key}[v]$.

The edge $(\text{parent}[v^*], v^*)$ is the *minimum weight edge crossing this cut*.

Cut Property $\Rightarrow$ this edge belongs to every MST.

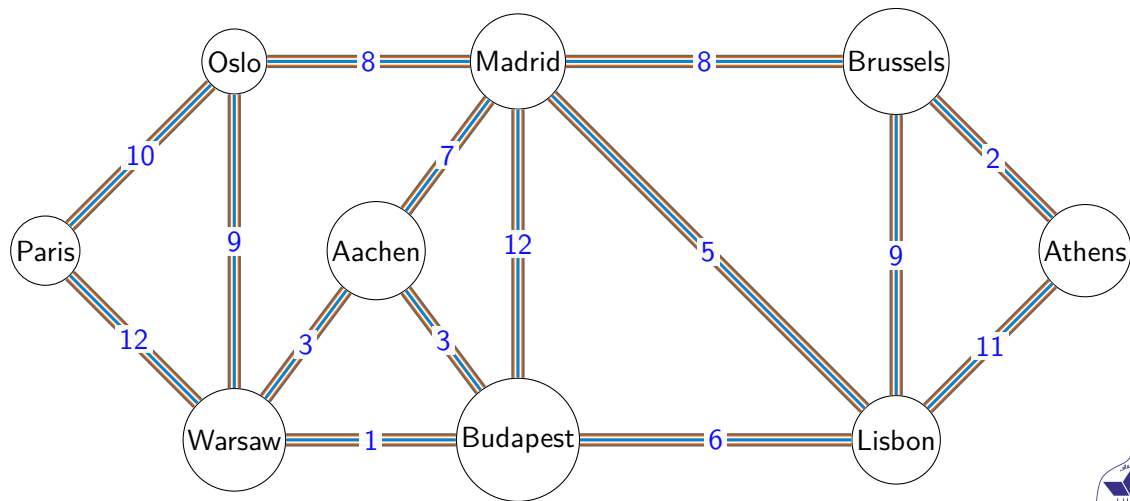
Every greedy step is safe. *Prim's is correct.*



$\text{key}[v^*] = 3$: minimum crossing edge.

Cut Property $\Rightarrow$ **safe to add.**

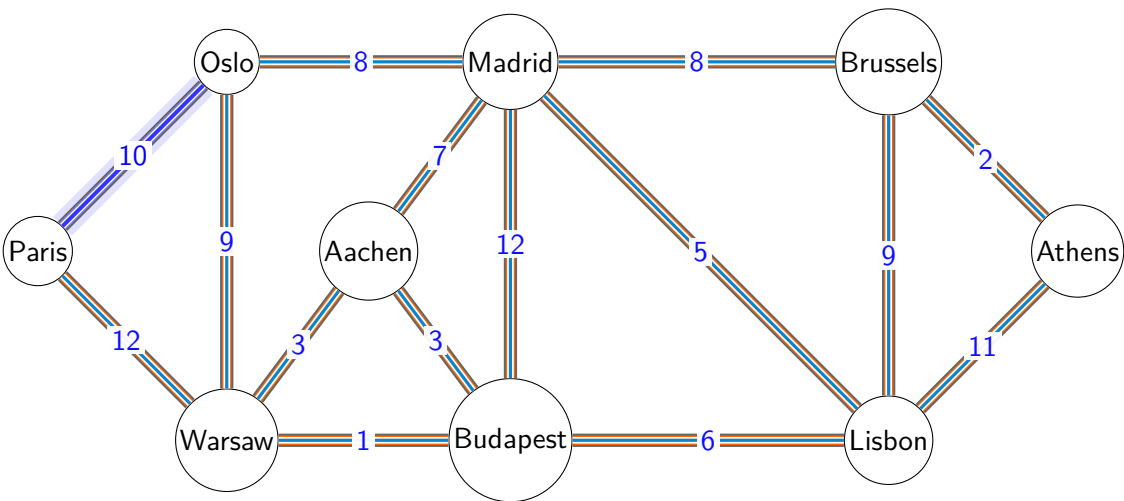
Prim's in Action



Starting from **Paris (A)**.

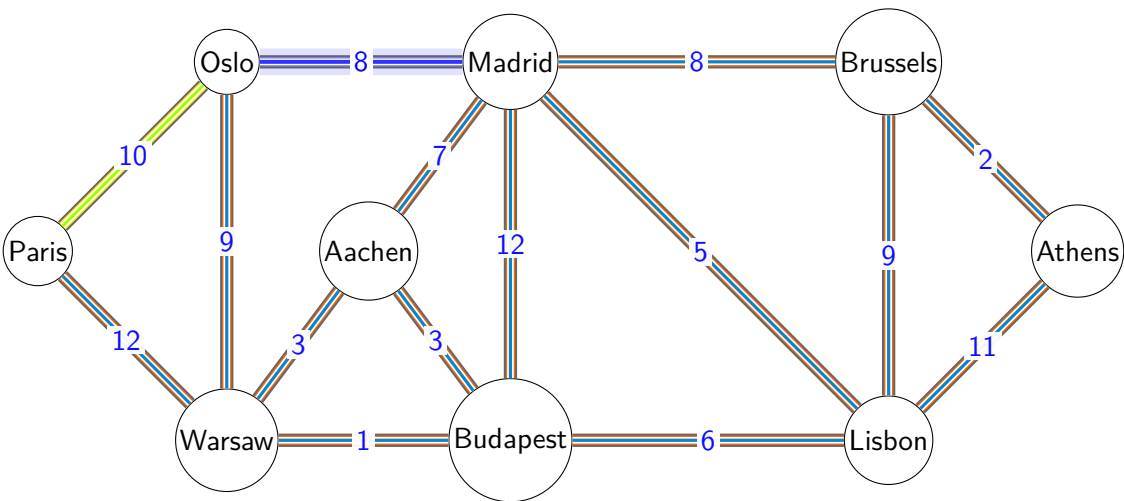
Adding Oslo (B) via AB (weight 10)

Frontier from Paris: AB(10), AC(12) — AB is minimum



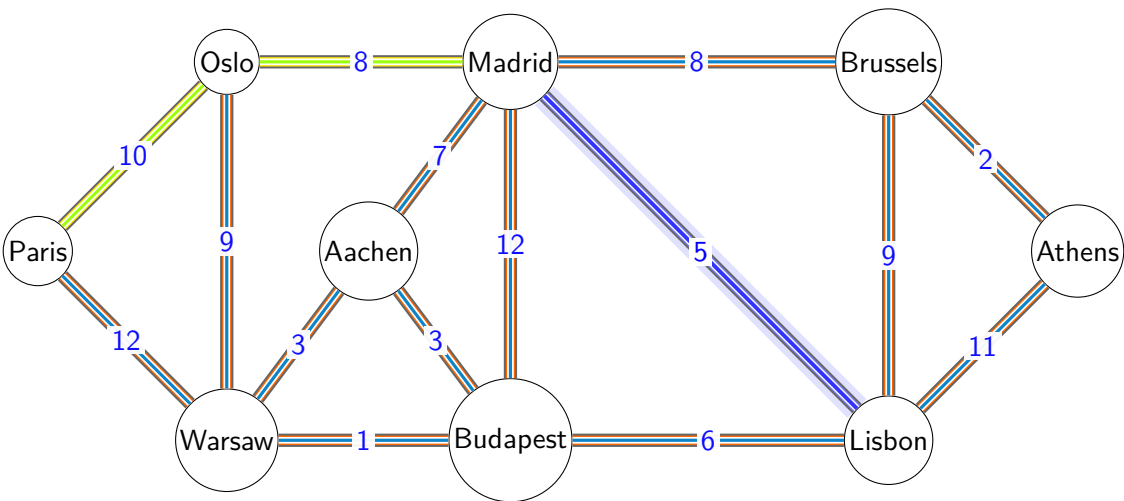
Adding Madrid (D) via BD (weight 8)

Frontier: BC(9), BD(8) — BD is minimum



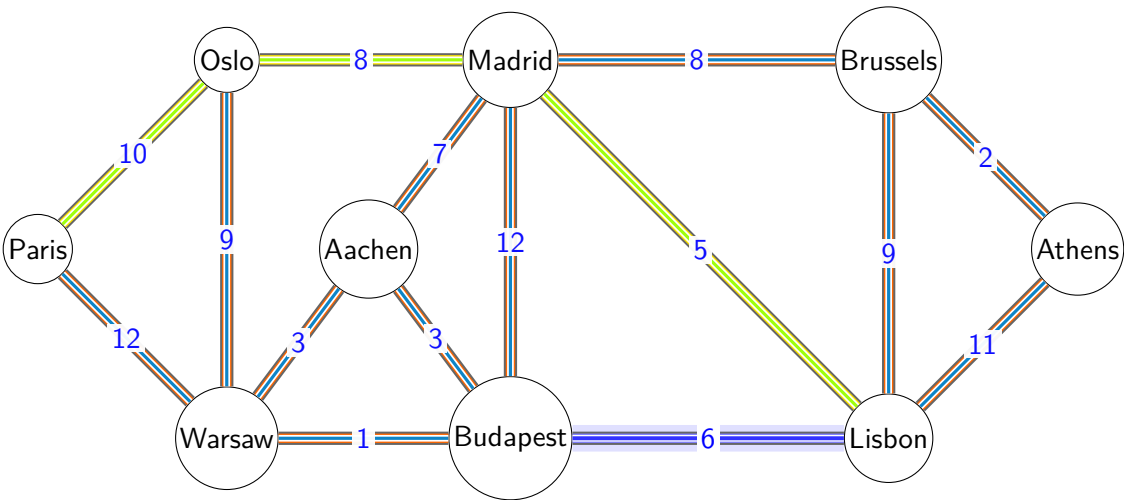
Adding Lisbon (H) via DH (weight 5)

Frontier: BC(9), DE(7), DH(5), DG(8), DF(12) — DH is minimum



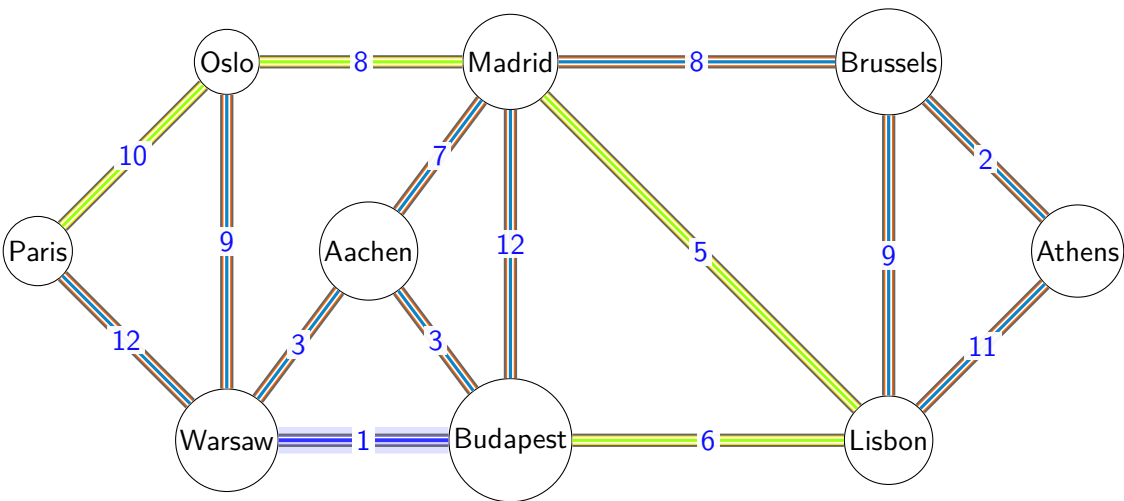
Adding Budapest (F) via FH (weight 6)

Frontier: BC(9), DE(7), FH(6), DG(8), HI(11) — FH is minimum

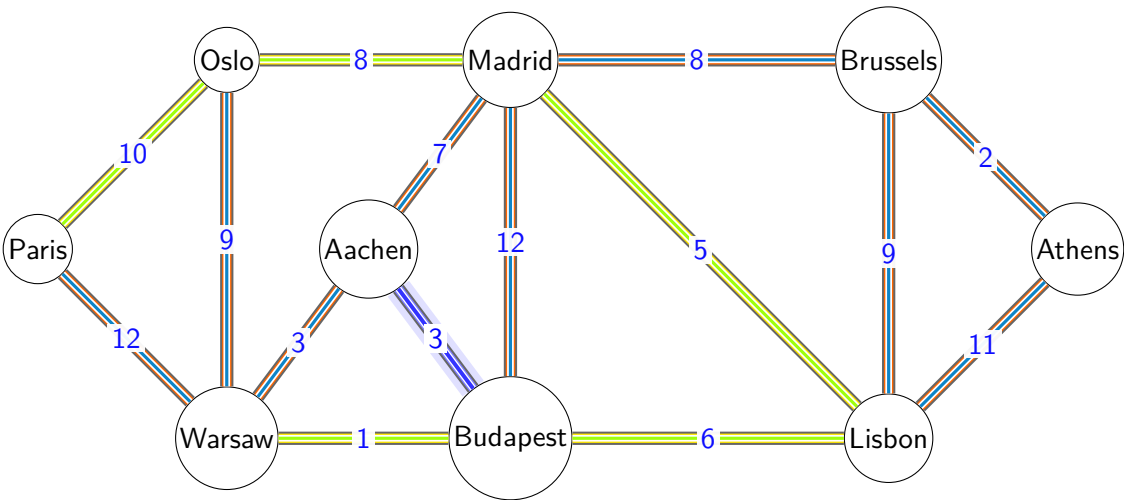


Adding Warsaw (C) via CF (weight 1)

Frontier: CF(1), EF(3), DG(8), HI(11) — CF is minimum

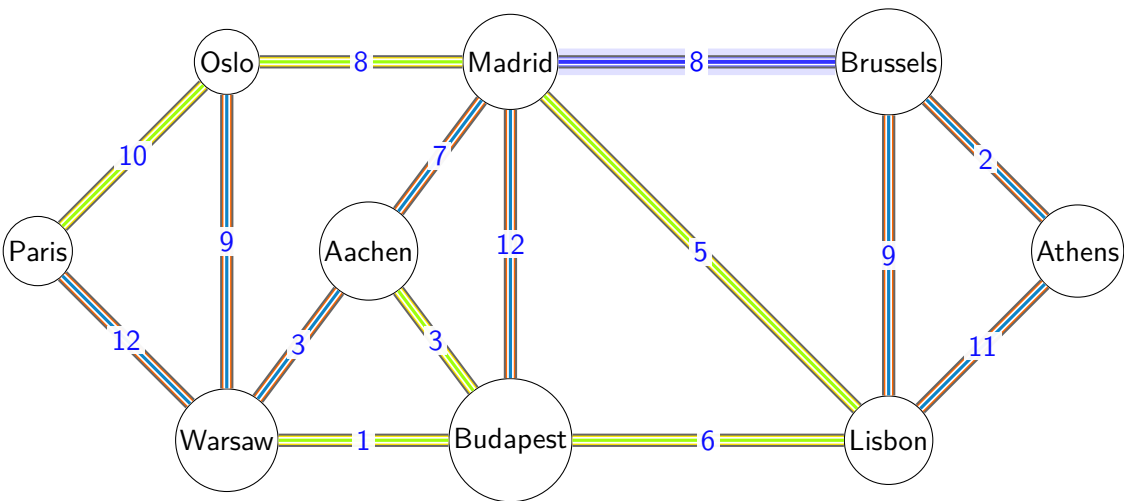


Adding Aachen (E) via EF (weight 3)
Frontier: EF(3), DG(8), HI(11) — EF is minimum



Adding Brussels (G) via DG (weight 8)

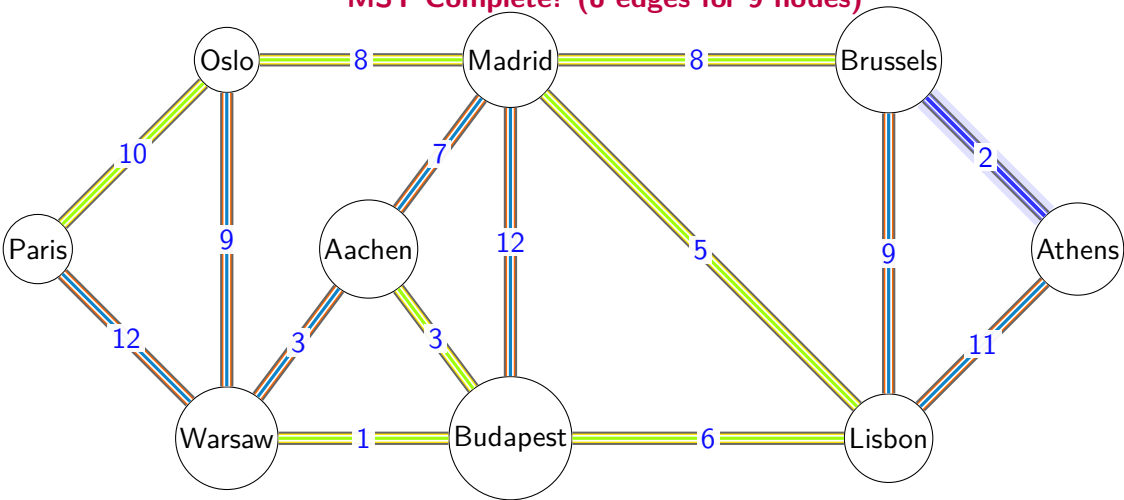
Frontier: DG(8), HI(11) — DG is minimum



Adding Athens (I) via GI (weight 2)

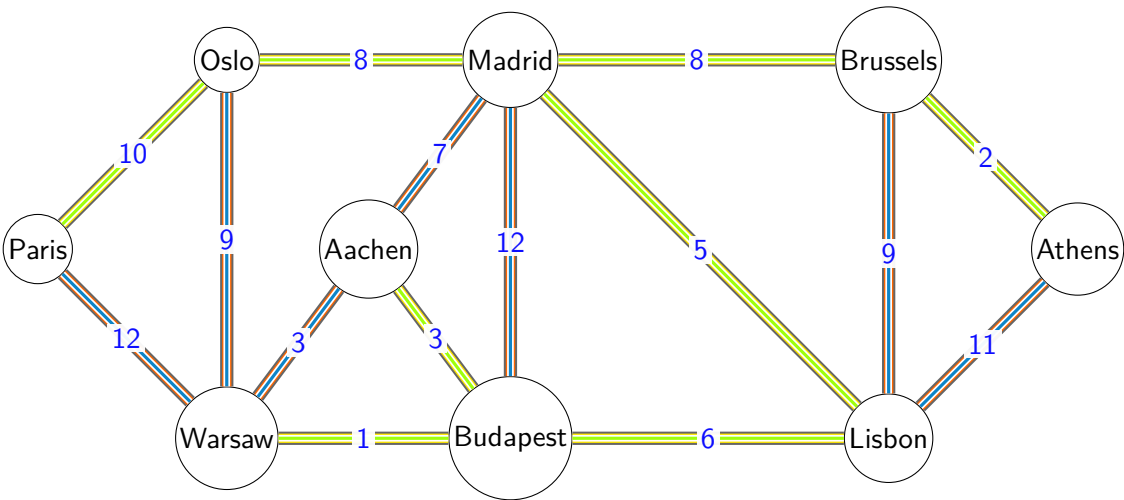
Frontier: GI(2) — last vertex

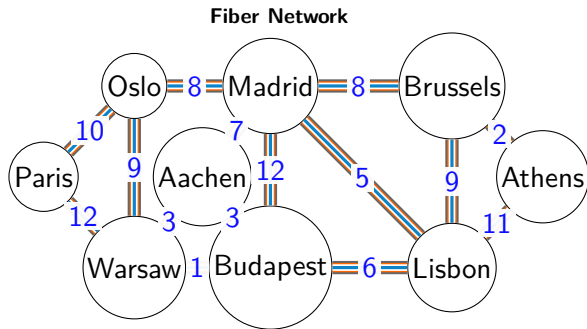
MST Complete! (8 edges for 9 nodes)



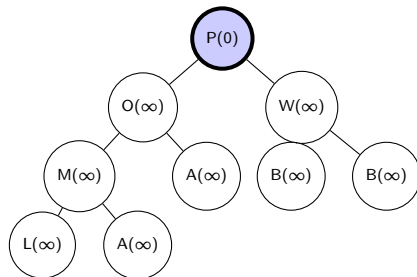
Minimum Spanning Tree

Total Weight: $1+2+3+5+6+8+8+10 = 43$

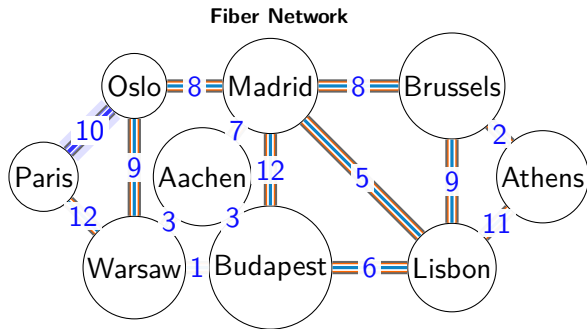




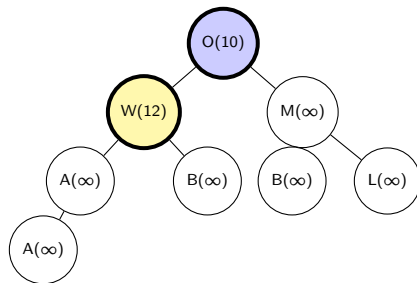
Priority Queue (Min-Heap)



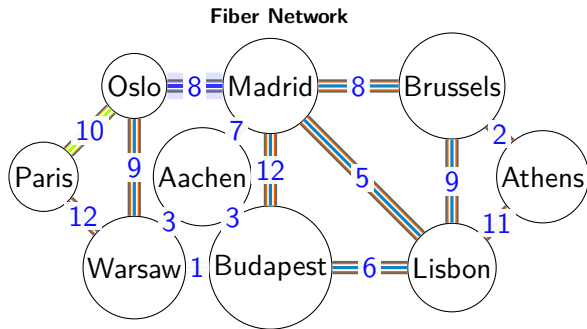
Tree vertex P extracted



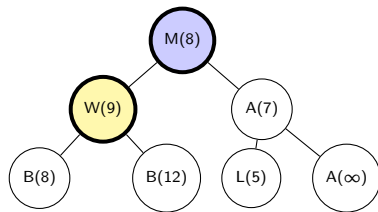
Priority Queue After Update



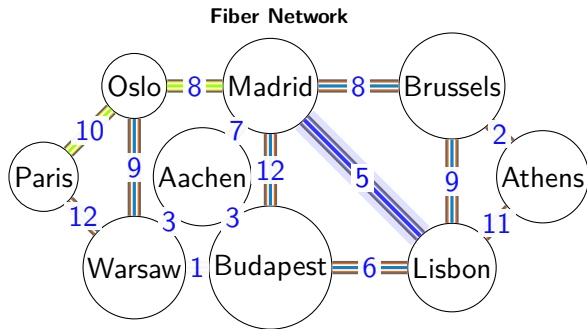
O will be next (min key 10)



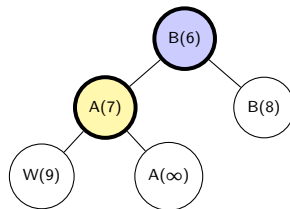
Priority Queue After Update



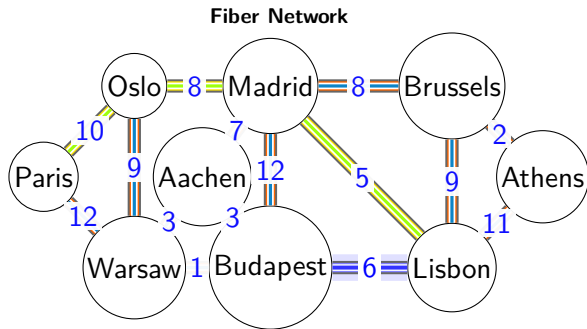
L will be next (min key 5)



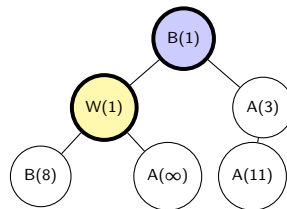
Priority Queue After Update



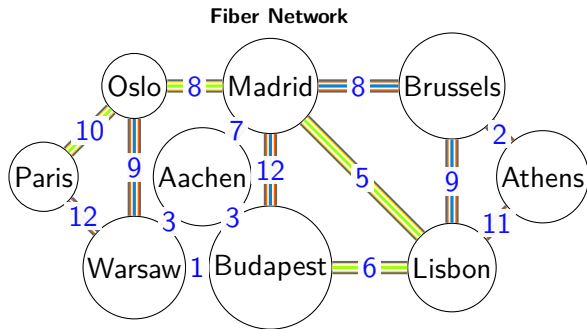
B will be next (min key 6)



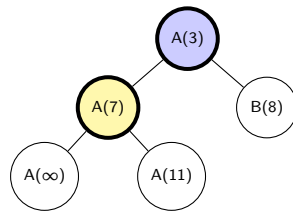
Priority Queue After Update



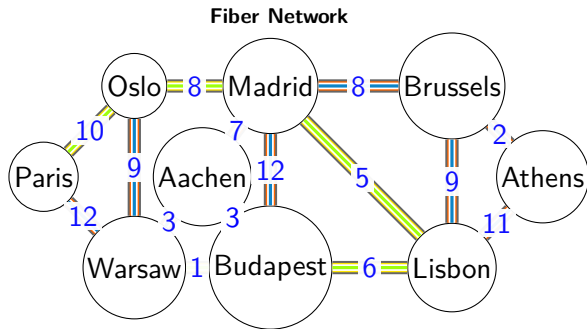
W will be next (min key 1)



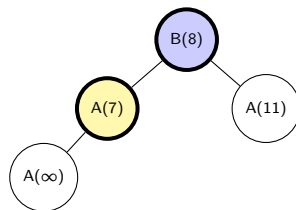
Priority Queue After Update



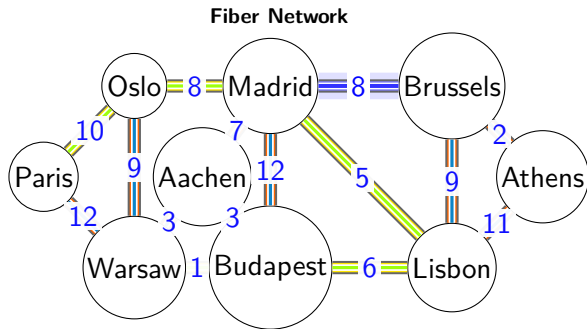
A will be next (min key 3)



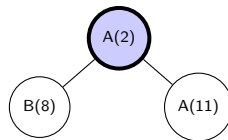
Priority Queue After Update



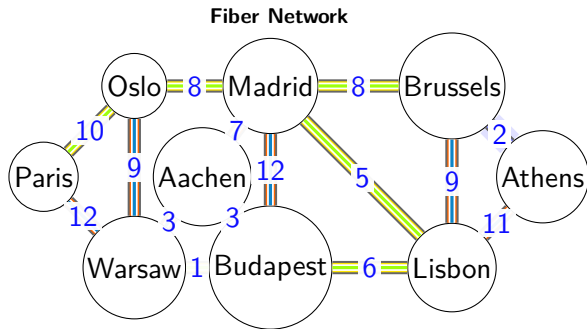
A will be next (min key 7)



Priority Queue After Update



A will be next (min key 2)



MST Complete

All vertices extracted

Total MST Weight: 43

Edges: AB(10), BD(8), DH(5)
 FH(6), CF(1), EF(3)
 DG(8), GI(2)

Prim's Time Complexity: The Priority Queue

The main loop runs V times (one ExtractMin per vertex).

Each edge is examined once, triggering at most one DecreaseKey.



Prim's Time Complexity: The Priority Queue

The main loop runs V times (one ExtractMin per vertex).

Each edge is examined once, triggering at most one DecreaseKey.

Simple array (unsorted list of non-tree vertices):

- ExtractMin: scan all entries.
- DecreaseKey: update one cell.

$O(V)$

$O(1)$



Prim's Time Complexity: The Priority Queue

The main loop runs V times (one ExtractMin per vertex).

Each edge is examined once, triggering at most one DecreaseKey.

Simple array (unsorted list of non-tree vertices):

- ExtractMin: scan all entries.

$O(V)$

- DecreaseKey: update one cell.

$O(1)$

- Total: $V \cdot O(V) + E \cdot O(1)$

$= O(V^2)$



Prim's Time Complexity: The Priority Queue

The main loop runs V times (one ExtractMin per vertex).

Each edge is examined once, triggering at most one DecreaseKey.

Simple array (unsorted list of non-tree vertices):

- ExtractMin: scan all entries.

$O(V)$

- DecreaseKey: update one cell.

$O(1)$

- Total: $V \cdot O(V) + E \cdot O(1)$

$= O(V^2)$

Binary heap:

- Both ExtractMin and DecreaseKey: $O(\log V)$.



Prim's Time Complexity: The Priority Queue

The main loop runs V times (one ExtractMin per vertex).

Each edge is examined once, triggering at most one DecreaseKey.

Simple array (unsorted list of non-tree vertices):

- ExtractMin: scan all entries. $O(V)$
- DecreaseKey: update one cell. $O(1)$
- Total: $V \cdot O(V) + E \cdot O(1)$ $= O(V^2)$

Binary heap:

- Both ExtractMin and DecreaseKey: $O(\log V)$.
- Total: $V \cdot O(\log V) + E \cdot O(\log V)$ $= O(E \log V)$



Prim's Time Complexity: The Priority Queue

The main loop runs V times (one ExtractMin per vertex).

Each edge is examined once, triggering at most one DecreaseKey.

Simple array (unsorted list of non-tree vertices):

- ExtractMin: scan all entries. $O(V)$
- DecreaseKey: update one cell. $O(1)$
- Total: $V \cdot O(V) + E \cdot O(1)$ $= O(V^2)$

Binary heap:

- Both ExtractMin and DecreaseKey: $O(\log V)$.
- Total: $V \cdot O(\log V) + E \cdot O(\log V)$ $= O(E \log V)$

Fibonacci heap:

- ExtractMin: $O(\log V)$ amortized. DecreaseKey: $O(1)$ amortized.



Prim's Time Complexity: The Priority Queue

The main loop runs V times (one ExtractMin per vertex).

Each edge is examined once, triggering at most one DecreaseKey.

Simple array (unsorted list of non-tree vertices):

- ExtractMin: scan all entries. $O(V)$
- DecreaseKey: update one cell. $O(1)$
- Total: $V \cdot O(V) + E \cdot O(1)$ $= O(V^2)$

Binary heap:

- Both ExtractMin and DecreaseKey: $O(\log V)$.
- Total: $V \cdot O(\log V) + E \cdot O(\log V)$ $= O(E \log V)$

Fibonacci heap:

- ExtractMin: $O(\log V)$ amortized. DecreaseKey: $O(1)$ amortized.
- Total: $V \cdot O(\log V) + E \cdot O(1)$

$$= O(E + V \log V)$$



Kruskal's vs Prim's: When to Use Which



Kruskal's vs Prim's: When to Use Which

Graph	$ E $	Kruskal's	Prim's (binary heap)	Prim's (Fib. heap)
Sparse	$O(V)$	$O(V \log V)$	$O(V \log V)$	$O(V \log V)$
Dense	$O(V^2)$	$O(V^2 \log V)$	$O(V^2 \log V)$	$O(V^2)$



Kruskal's vs Prim's: When to Use Which

Graph	$ E $	Kruskal's	Prim's (binary heap)	Prim's (Fib. heap)
Sparse	$O(V)$	$O(V \log V)$	$O(V \log V)$	$O(V \log V)$
Dense	$O(V^2)$	$O(V^2 \log V)$	$O(V^2 \log V)$	$O(V^2)$

- Sparse graphs: Kruskal's and Prim's (binary heap) are equivalent.
- Dense graphs: Prim's with a Fibonacci heap (or even a simple array: $O(V^2)$) wins over Kruskal's.



Kruskal's vs Prim's: When to Use Which

Graph	$ E $	Kruskal's	Prim's (binary heap)	Prim's (Fib. heap)
Sparse	$O(V)$	$O(V \log V)$	$O(V \log V)$	$O(V \log V)$
Dense	$O(V^2)$	$O(V^2 \log V)$	$O(V^2 \log V)$	$O(V^2)$

- Sparse graphs: Kruskal's and Prim's (binary heap) are equivalent.
- Dense graphs: Prim's with a Fibonacci heap (or even a simple array: $O(V^2)$) wins over Kruskal's.

In practice

Kruskal's is usually simpler to implement correctly.

Prim's advantage is mainly relevant for dense graphs.